

Learning Objective

By the end of this session, you'll understand:

1. What **Azure SQL** is and its **deployment options**
 2. How to **set up an Azure SQL Database** (step-by-step using Azure Portal)
 3. How to **connect** to it (using SQL Server Management Studio or Azure Data Studio)
 4. How to **secure and manage** it with firewall and authentication options
-

Part 1: Conceptual Understanding

What is Azure SQL?

Azure SQL is a **fully managed relational database service** built on **Microsoft SQL Server**. It's offered as part of the **Azure Data Platform** and designed for scalability, high availability, and security — without worrying about server maintenance.

Azure SQL Family

Azure offers **3 main options** depending on your control needs:

Option	Description	Use Case
1 Azure SQL Database (PaaS)	Fully managed single database	Ideal for modern applications with predictable workloads
2 Azure SQL Managed Instance (PaaS)	Almost full SQL Server instance with minimal admin	For migrating existing on-prem SQL databases to Azure easily
3 SQL Server on Azure VM (IaaS)	Full SQL Server installed on an Azure Virtual Machine	When you need OS-level control or custom SQL configurations

Key Features

- ✓ Automatic backups and point-in-time restore
- ✓ Built-in high availability (99.99% uptime SLA)
- ✓ Security (Transparent Data Encryption, Firewall, Private Link)

- ✓ Scaling (DTUs or vCores)
 - ✓ Integration with Azure services like Key Vault, Logic Apps, and Functions
-

Part 2: Step-by-Step — Creating Azure SQL Database (Using Azure Portal)

Let's go through a **hands-on example** of setting up a **single Azure SQL Database**.

Step 1: Open Azure Portal

- Navigate to  <https://portal.azure.com>
 - Sign in with your Azure account
-

Step 2: Create a SQL Database

1. Click **Create a resource** → Search “SQL Database”
2. Click **Create**

You'll see 4 configuration tabs:

- **Basics**
 - **Networking**
 - **Security**
 - **Additional Settings**
-

Step 3: Configure Basics

Fill the form as follows:

Field	Example
Subscription	Your Azure subscription
Resource Group	rg-sql-demo
Database name	sqldemodb

Field	Example
Server	Click “Create new”
Server name	sqlserverdemo123 (must be unique globally)
Server admin login	sqladmin
Password	Strong password (e.g., P@ssword12345!)
Location	Select a nearby region (e.g., East US / Central India)
Compute + storage	Select “Basic” or “General Purpose”
Click Next: Networking →	

Step 4: Configure Networking

- **Connectivity method:** Public endpoint (for demo)
- **Firewall rules:** Add your current client IP (so you can connect later)
- **Connection policy:** Default

Click **Next: Security** →

Step 5: Configure Security

- Enable **Microsoft Defender for SQL** (optional for demos)
- Enable **Transparent Data Encryption (TDE)** (default is ON)

Click **Review + Create** and then **Create**

Step 6: Deployment

Deployment will take 2–4 minutes.

Once complete, go to your **resource overview page**.

You’ll find:

- **Server name:** sqlserverdemo123.database.windows.net

- **Database name:** sqldemodb
 - **Connection strings:** ADO.NET / JDBC / ODBC formats available
-

Part 3: Connect to Azure SQL Database

You can connect using **SQL Server Management Studio (SSMS)** or **Azure Data Studio**.

Steps (Using SSMS)

1. Open **SSMS**
2. Click **Connect → Database Engine**
3. Enter:
 - **Server name:** sqlserverdemo123.database.windows.net
 - **Authentication:** SQL Server Authentication
 - **Login:** sqladmin
 - **Password:** (the one you created)
4. Click **Connect**

 You're now connected to your Azure SQL database!

You can run queries like:

```
CREATE TABLE Products (  
    Id INT IDENTITY(1,1) PRIMARY KEY,  
    Name NVARCHAR(100),  
    Price DECIMAL(10,2)  
);  
  
INSERT INTO Products (Name, Price)  
VALUES ('Laptop', 75000), ('Mouse', 1200);  
  
SELECT * FROM Products;
```

Part 4: Security & Management

Security Options

- **Firewall rules** — Restrict access by IP address
 - **Azure AD Authentication** — Use Azure AD users instead of SQL logins
 - **Private Endpoints** — Connect through private network instead of public internet
 - **Key Vault integration** — Store connection strings & secrets securely
-

Scaling & Monitoring

- You can **scale up or down** (increase DTUs or vCores) without downtime
 - Use **Query Performance Insight** and **Metrics** in the portal to monitor performance
-

Real-world Integration Example

In an **ASP.NET Core API**, you can connect Azure SQL like this:

```
{  
  "ConnectionStrings": {  
    "DefaultConnection": "Server=tcp:sqlserverdemo123.database.windows.net,1433;Initial  
Catalog=sqldemodb;Persist Security Info=False;User  
ID=sqladmin;Password=P@ssword12345!;MultipleActiveResultSets=False;Encrypt=True;TrustSer  
verCertificate=False;Connection Timeout=30;"  
  }  
}
```

and in Program.cs:

```
builder.Services.AddDbContext<AppDbContext>(options =>  
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));
```

Summary

Concept	Description
---------	-------------

Azure SQL	Fully managed SQL Server database service
------------------	---

Types	SQL Database, Managed Instance, SQL VM
--------------	--

Setup Steps	Create → Configure Server → Set Firewall → Connect
--------------------	--

Security	TDE, Firewall, Private Link, Azure AD Auth
-----------------	--

Integration	Easy with .NET, Python, Java, Node.js
--------------------	---------------------------------------